Practical-2:

1. Create a JSON object and display the same using node.js in the terminal.
2. Create a Node.js script to read and display the content of the external JSON file.
3. Create and display multi-dimensional JSON Arrays, also demonstrate Accessing individual elements of the multi-dimensional JSON Arrays.
4. Create a web-based application example using JavaScript and JSON to manipulate JSON data. In this example, create a simple to-do list application where tasks are stored in a JSON array. Users can add tasks, mark them as completed, and remove them from the list. [**Note:** Perform this practical by designing and implementing an application relevant to the project title identified by the student in **Practical-1**.]

---

1. To display the contents of a JSON object in Node.js, you can use the **console.log** function to print the object to the console. Here's a simple example:

File name**: display.js**

```
// Sample JSON object
const jsonObject = {
  name: 'Devendra Vashi',
  age: 46,
  city: 'Ahmedabad',
};

// Display JSON object content
console.log(jsonObject);
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\MCA-3\Desktop\Full Stack Development\Practical\Practical-2> node display.js
{ name: 'John Doe', age: 25, city: 'Example City' }
PS C:\Users\MCA-3\Desktop\Full Stack Development\Practical\Practical-2>
```

---

2. File name**: data.json**

```
{
  "name": "John Doe",
  "age": 25,
  "city": "Example City"
}
```

File name**: script.js**

```
// Node.js fs module, which provides an API for interacting with the file system.
const fs = require('fs');

// Specify the path to the JSON file
const filePath = 'C:/Users/MCA-3/Desktop/Full Stack
Development/Practical/Practical-2/data.json';
```

```
// Read the JSON file
fs.readFile(filePath, 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading the JSON file:', err);
    return;
  }

  try {
    // Parse the JSON data
    const jsonObject = JSON.parse(data);

    // Display the content
    console.log('JSON object content:', jsonObject);

    // If you want formatted output
   console.log('Formatted JSON object content:', JSON.stringify(jsonObject, null, 2));
  } catch (parseError) {
    console.error('Error parsing JSON:', parseError);
  }
});
```

The **fs.readFile** function is asynchronous and takes three parameters: the file path (filePath), the encoding ('utf8' for text-based files), and a callback function. The callback function is executed once the file is read. If there is an error, it prints an error message. If there is no error, the file content is available in the data parameter.

The **JSON.parse()** method is a built-in JavaScript method that parses a JSON string, transforming it into a JavaScript object. The method takes a JSON-formatted string as its argument (data in this case) and returns the equivalent JavaScript object.

The **JSON.stringify()** method is a built-in JavaScript method that converts a JavaScript object into a JSON-formatted string. This method has **three** parameters:

The **first** parameter is the object to be converted (jsonObject in this case).
The **second** parameter is a replacer function or an array that specifies how object values should be stringified, **or** null if no transformation is needed.
The **third** parameter is the number of spaces to use for indentation when formatting the JSON string (in this case, 2 spaces).

```
PS C:\Users\MCA-3\Desktop\Full Stack Development\Practical\Practical-2> node script.js
JSON object content: { name: 'John Doe', age: 25, city: 'Example City' }
Formatted JSON object content: {
  "name": "John Doe",
  "age": 25,
  "city": "Example City"
}
```

3. File name**: multi-dimensional-JSON-display.js**

```
// Create a multi-dimensional JSON array
const multiDimArray = [
```

```
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
  ['apple', 'banana', 'orange']
];

// Display the multi-dimensional JSON array
console.log('Multi-dimensional JSON Array:');
console.log(multiDimArray);

// Accessing individual elements
console.log('\nAccessing individual elements:');
console.log('Element at [0][1]:', multiDimArray[0][1]); // Output: 2
console.log('Element at [2][2]:', multiDimArray[2][2]); // Output: 9
console.log('Element at [3][0]:', multiDimArray[3][0]); // Output: 'apple'
```

```
PS C:\Users\MCA-3\Desktop\Full Stack Development\Practical\Practical-2> node multi-dimensional-JSON-display.js
Multi-dimensional JSON Array:
[
  [ 1, 2, 3 ],
  [ 4, 5, 6 ],
  [ 7, 8, 9 ],
  [ 'apple', 'banana', 'orange' ]
]

Accessing individual elements:
Element at [0][1]: 2
Element at [2][2]: 9
Element at [3][0]: apple
```

4. File name**: manipulated-JSONdata.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Todo List App</title>

<style>
    ul {
      list-style-type: none;
      padding: 0;
    }

    li {
      margin: 5px 0;
    }

    .completed {
      text-decoration: line-through;
    }
</style>

</head>
<body>
```

```
<h1>Todo List App</h1>

<input type="text" id="taskInput" placeholder="Add a new task">
<button onclick="addTask()">Add Task</button>

<ul id="taskList"></ul>

<script>
  // Sample JSON array for tasks (initializes a JavaScript array named tasks)
  let tasks = [
    { id: 1, text: 'Complete assignment', completed: false },
    { id: 2, text: 'Read a book', completed: true },
    { id: 3, text: 'Go for a run', completed: false }
  ];

  // Function to display tasks on the web page
  function displayTasks() {
    const taskListElement = document.getElementById('taskList');
    taskListElement.innerHTML = ' '; // By doing this, any existing content within the
                                        'taskList' element is completely removed or cleared.

    tasks.forEach(task => {
      const li = document.createElement('li');
      li.textContent = task.text;
      if (task.completed) {
        li.classList.add('completed');
      }

      li.onclick = () => toggleTaskStatus(task.id);
      taskListElement.appendChild(li);
    });
  }

  // Function to add a new task
  function addTask() {
    const taskInput = document.getElementById('taskInput');
    const newTaskText = taskInput.value.trim();

    if (newTaskText !== ' ') {
      const newTask = {
        id: tasks.length + 1,
        text: newTaskText,
        completed: false
      };

      tasks.push(newTask);
      displayTasks();
      taskInput.value = '';
    }
  }

  // Function to toggle the status of a task (completed or not)
```

```
  function toggleTaskStatus(taskId) {
    const taskIndex = tasks.findIndex(task => task.id === taskId);

    if (taskIndex !== -1) {
      tasks[taskIndex].completed = !tasks[taskIndex].completed;
      displayTasks();
    }
  }

  // Initial display of tasks
  displayTasks();
 </script>

</body>
</html>
```

width=device-width:
This part sets the width of the viewport to the width of the device. In other words, it ensures that the width of the web page is equal to the width of the device's screen.

list-style-type: none; removes the default list-style (bullet points or numbering) from the unordered list.
padding: 0; sets the padding of the unordered list to zero, removing any default spacing.
margin: 5px 0; sets a margin of 5 pixels above and below each list item. This creates spacing between adjacent list items.
text-decoration: line-through; applies a line-through decoration to the text of elements with the class "completed." This is often used to visually indicate that an item, such as a to-do list item, has been completed.

